University of Basel

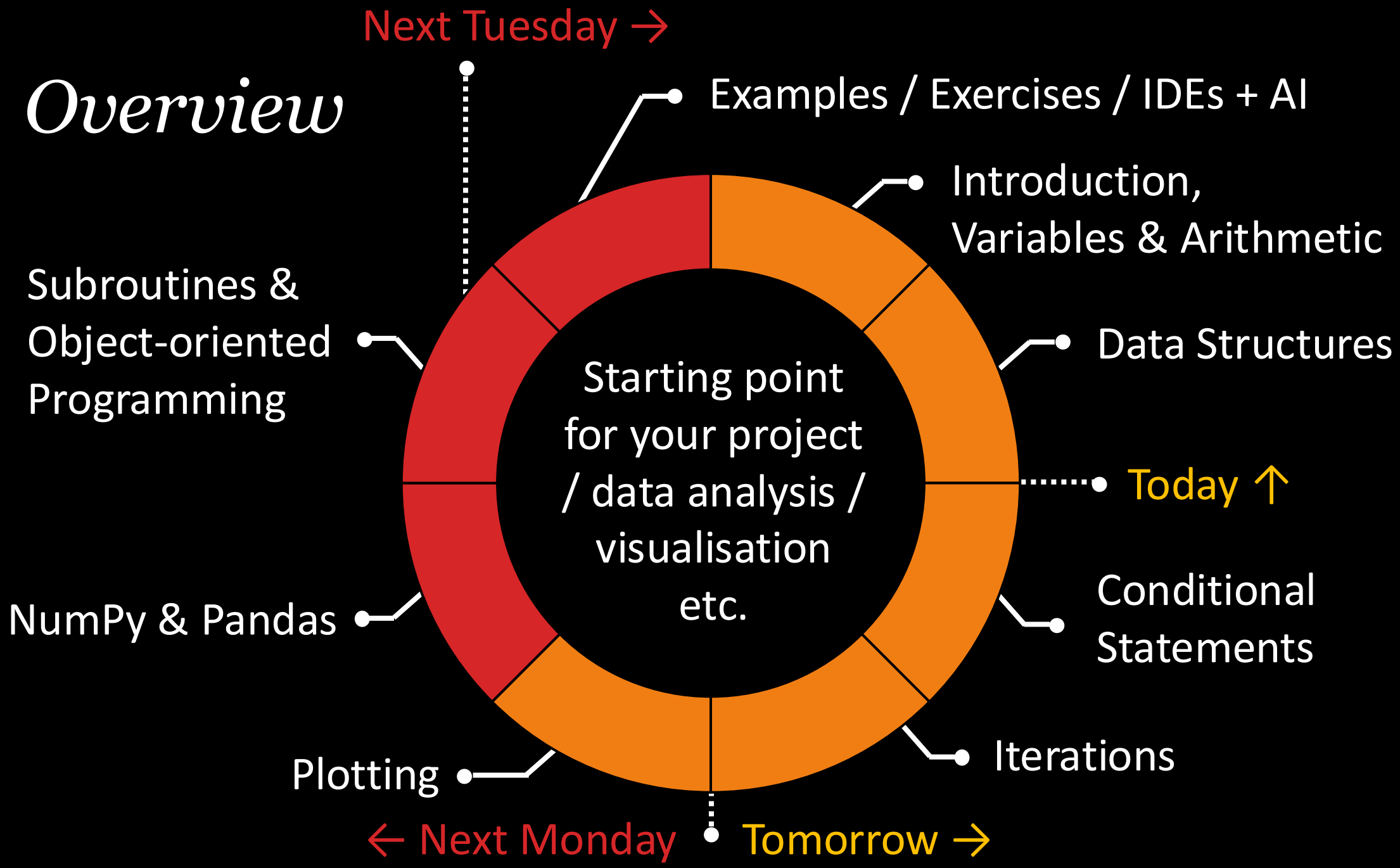
SDSC

Python Crash Course

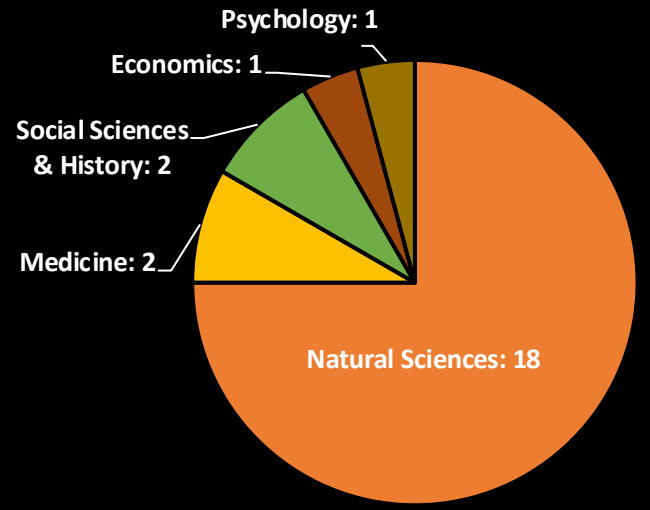
Dr. Maxim Samarin
Senior Data Scientist @ Swiss Data Science Center

5th / 6th / 12th / 13th October 2026

Overview


Next Tuesday →
Examples / Exercises / IDEs + AI
Introduction,
Variables & Arithmetic
Data Structures
Today ↑
Conditional
Statements
Iterations
Tomorrow →
← Next Monday
Plotting
NumPy & Pandas
Subroutines &
Object-oriented
Programming
Starting point
for your project
/ data analysis /
visualisation
etc.

Your Experience and Goal



- What is your programming experience so far and an intermediate goal for you?

Alaa ⇢ Asia ⇢ Gabriele ⇢ Alessandro ⇢ Harshithaa ⇢ ⇢ Helen G. ⇢ Gariel ⇢ Noor ⇢ Rahel ⇢ Tamanna ⇢ ⇢ Rory ⇢ Giulia ⇢ Manuela ⇢ Xiya ⇢ Cassandra ⇢ ⇢ Lilian ⇢ Margherita ⇢ Elia ⇢ Narmadha ⇢ Victoria ⇢ ⇢ Mai ⇢ Heleen V. ⇢ Adrien ⇢ Simone

My Programming Career



- School classes programming in Java
- Studying Physics: C++, MatLab, Python
- Project: Analysis & Graphical User Interface in C, Python
- Physics modelling in Fortran, Python
- M.Sc. thesis: Data Mining & simulations in R, Python
- Ph.D. thesis: Machine / Deep Learning research in Python ...
- Data Scientist and ML / AI Researcher : Python ...

Python vs. other Languages



- Python is a dynamically typed programming (script) language
- Code is interpreted
- A lot of details *under the hood*
- Zen of Python: Simple, explicit, sparse, readable, practical

→ Fast prototyping, quick to learn





- Statically typed: fix type and object
- Code is compiled
- A lot of explicit control
- Efficient implementations, elaborate syntax

→ Fast and powerful codes

Python & C++ Examples






```python
name = input("Please enter your name: ")
print("Good morning,", name)
```

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string name;

    cout << "Please enter your name: ";
    cin >> name;
    cout << "Good morning, " << name << endl;

    return 0;
}
```

- Both have the same output:

  Please enter your name: Monty
  Good morning, Monty

Start your Working Environment

1. Access the course environment via Renku link, sign in with your SWITCH edu-ID and launch the session
2. Or: Download new material, start Jupyter Lab and open notebooks

- Suggestions on how to work in this course:

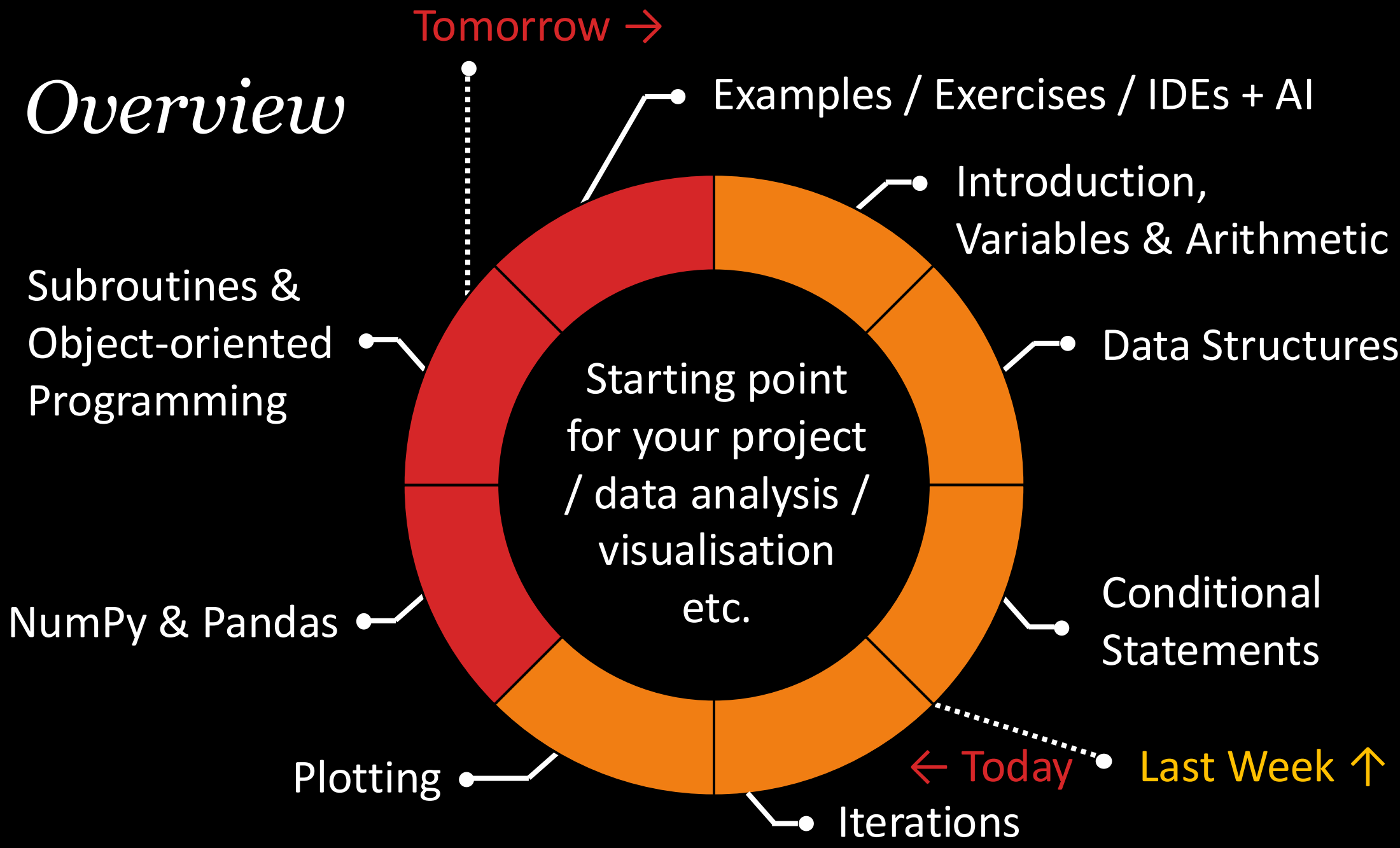


: Follow presentation, while executing scripts yourself, making adjustments and notes in your own notebook



: Follow presentation, switch to programming environment for exercises

Overview


Tomorrow →
Examples / Exercises / IDEs + AI
Introduction,
Variables & Arithmetic
Data Structures
Conditional
Statements
← Today
Last Week ↑
Iterations
Plotting
NumPy & Pandas
Subroutines &
Object-oriented
Programming
Starting point
for your project
/ data analysis /
visualisation
etc.

Different Ways to Execute Scripts

- Jupyter Notebooks (as in this class): Document-style, combining executable scripts with rich documentation and formatting
  → Markdown and LaTeX can be used
- Terminal: Execute scripts via the terminal
  1. Write your script in an editor and save it to a *my_script.py* file
  2. Execute the script in the terminal with *python my_script.py*
- Spyder (beginner friendly): Integrated development environment (IDE) → editor with a terminal and other useful functionalities
- Or any other editors / IDE such as Atom, Sublime Text, PyCharm etc.

Advanced Setup: IDEs & Copilot

- Visual Studio Code (VS Code): Free for all
  - Installation: https://code.visualstudio.com/
- PyCharm Professional: Free for students (Ph.D.) & teachers (Postdoc)
  - Installation: https://www.jetbrains.com/pycharm/
  - License: https://www.jetbrains.com/community/education/#students
- GitHub Copilot Pro: Free for students (Ph.D.) & teachers (Postdoc)
  - Get a certificate of employment (”Arbeitsbestätigung”) from HR (in EN)
  - Documentation for license
    - PhD student: https://docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-students
    - Postdoc: https://docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-teachers-and-os-maintainers

Selection of References

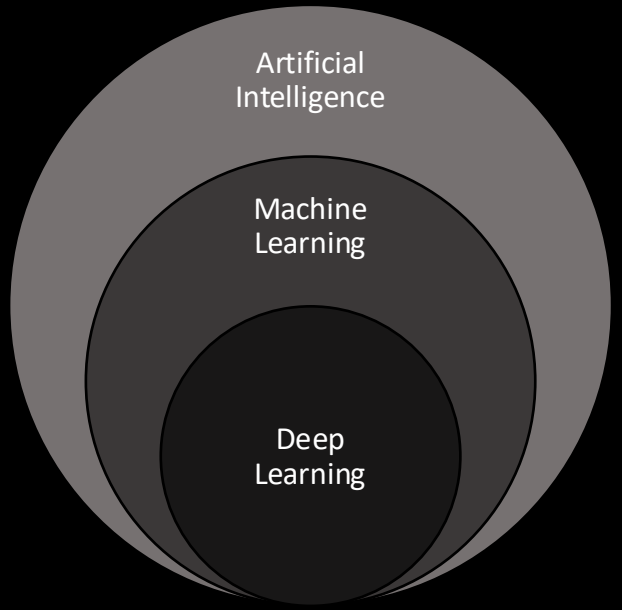
- Quick overview with interactive tutorials on some basic topics (similar to the course) and more advanced concepts: https://www.learnpython.org/
- More detailed overview with interactive tutorials on a lot of topics: https://www.w3schools.com/python/default.asp
- One of my favourites with tutorials on specific (advanced) topics, easy-to-read books: https://www.realpython.com/
- Overview of resources for beginners: https://wiki.python.org/moin/BeginnersGuide

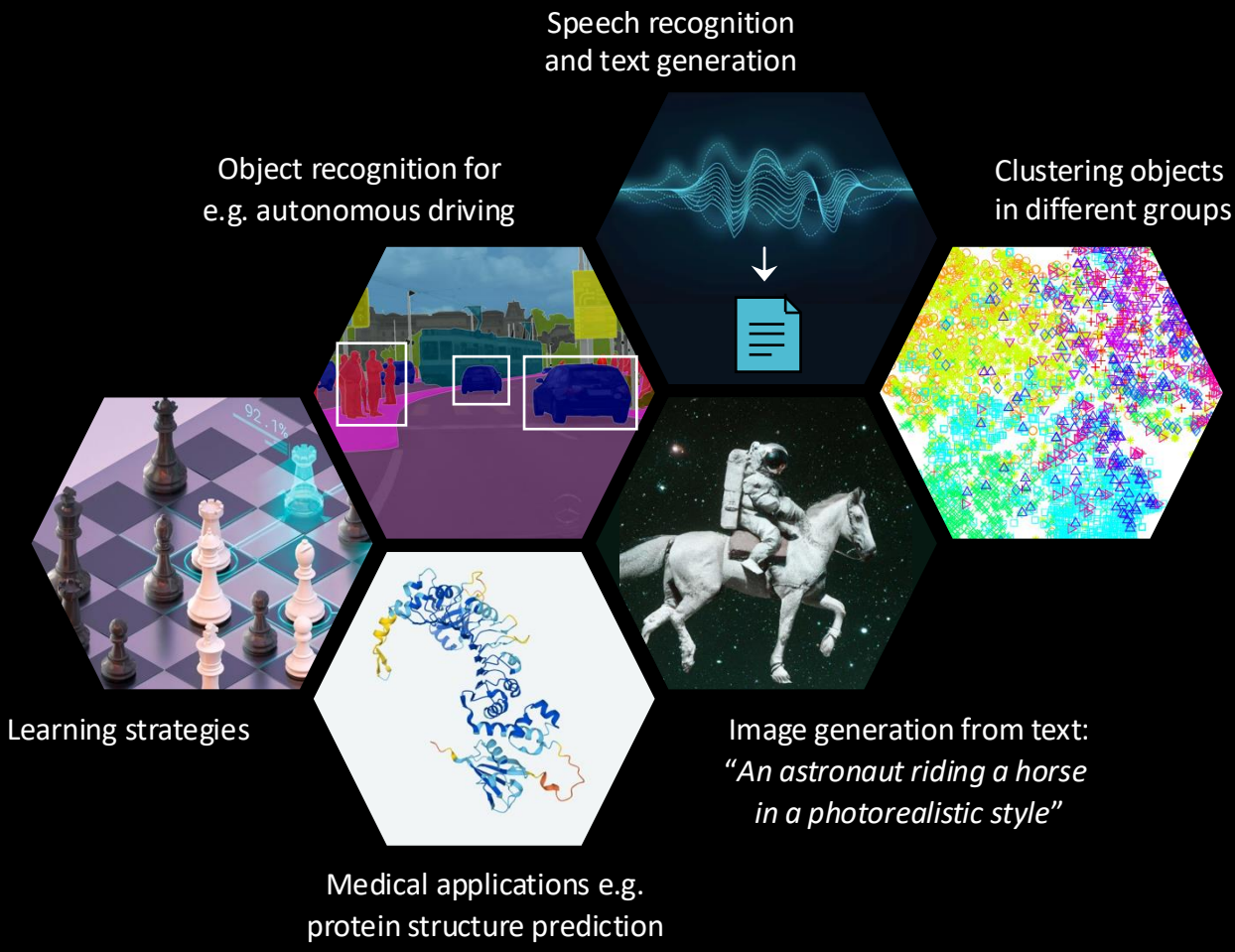
AI-Assisted Coding & Modern Workflows

- Next course (in January 2027):
  - Overview of different AI tools with explicit examples
    - working with documents and .md files
    - creating visuals / images
    - focus on programming in Python
    - covering tools like NotebookLM, ChatGPT, Claude Code, Adobe Firefly, OpenAI Codex
  - High-level training mechanisms of LLMs
  - Introduction of classical programming toolset: Git, command-line interface, IDEs
  - From AI-assisted coding to “vibe coding” with GitHub Copilot

Advanced Python & Machine Learning

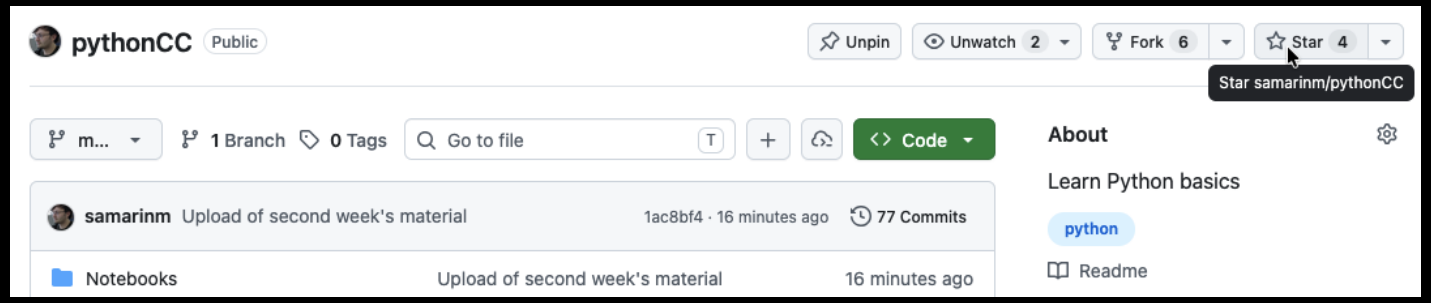
- Next course (next spring):
  - More advanced Python concepts
  - Introduction to Machine Learning





Thank you and good luck as new Pythonistas! ☺

Your feedback:
https://evasys.unibas.ch/evasys/online.php?p=7FL73



Consider giving a ★ if you liked the course!

Some Additional Links

- Download the course material from https://github.com/samarinm/pythonCC
- On script languages: https://en.wikipedia.org/wiki/Scripting_language
- On the difference between interpreter and compiler: https://www.programiz.com/article/difference-compiler-interpreter
- On dynamic and static type checking: https://en.wikipedia.org/wiki/Type_system#Type_checking